that maps column names in the database to safe Ruby method names. You could certainly foresee special cases such as these, but catching them all could prove very hard. It's arguably easier to cover a reasonable number of special cases like Rails 1 did, and then change your code as more special cases become visible.

Rails' approach seems to be very much biased toward Evolutionary Design rather than Do It Right the First Time. There are two obvious reasons for that. First, Ruby is a flexible, pliable language, especially when you use metaprogramming, so it's generally easy to evolve your code as you go. And second, writing perfect metaprogramming code up front can be hard, because it can be difficult to uncover every possible corner case.

To sum it all up in a single sentence: *keep your code as simple as possible, and add complexity as you need it*. When you start, strive to make your code correct in the general cases, and simple enough that you can add more special cases later. This is a good rule of thumb for most code, but it seems to be especially relevant when metaprogramming is involved.

This last consideration also leads us to a final, deeper lesson—one that has to do with the meaning of metaprogramming itself.